

**VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF APPLIED SCIENCE**



LINEAR ALGEBRA

Sem 251 - CC02 - Group 02

**PRINCIPAL COMPONENT ANALYSIS: DATA DIMENSIONALITY
REDUCTION**

Instructor: Phan Thi Khanh Van

Students:	Dao Tan Vy (Leader)	ID 2551458
	Nguyen Hoang Vinh Phuc	ID 2551446
	Nguyen Chi Duc	ID 2551425
	Nguyen Hoang Dang Khoa	ID 2452546
	Nguyen Nam Son	ID 2353051

Ho Chi Minh, 2025

TABLE OF CONTENTS

INTRODUCTION	1
CHAPTER 1: OVERVIEW	2
1.1 Problem Definition: Data Dimensionality	2
1.1.1 Definition of Data Dimensionality	2
1.1.2 The Curse of Dimensionality	2
1.1.3 Why Dimensionality Reduction is Needed	2
1.1.4 Types of Dimensionality Reduction Techniques	3
1.2 Introduction to PCA	3
1.2.1 What is Principal Component Analysis?	3
1.2.2 Historical Background	3
1.2.3 Key Intuition	3
1.2.4 Illustrative Example	3
1.2.5 Relationship with Other Methods	4
1.3 Objectives, Advantages, and Disadvantages	4
1.3.1 Main Objectives	4
1.3.2 Advantages	4
1.3.3 Disadvantages	5
CHAPTER 2: Mathematical Foundations	6
2.1 Statistical Features	6
2.1.1 Expectation	6
2.1.2 Variance	6
2.1.3 Standard Deviation	7
2.2 Covariance	7
2.2.1 Definition (Scalar)	7
2.2.2 Properties	7
2.2.3 Covariance Matrix (Multivariate)	7
2.2.4 Conditional Covariance and Law of Total Covariance	8
2.2.5 Whitening / Decorrelation	8
2.2.6 Difference Between Diagonal and Off-Diagonal Values	8
2.3 Eigenvalues and Eigenvectors	9
2.3.1 Algebraic Definition	9
2.3.2 Spectral Properties	9
2.3.3 Singular Value Decomposition (SVD)	10
2.3.4 Eigen-decomposition of Covariance Matrices & PCA	10
CHAPTER 3: The Principal Component Analysis Algorithm	11
3.1 Theoretical Framework	11
3.1.1 Step 1: Standardization (Data Scaling)	11

3.1.2	Step 2: Covariance Matrix Computation	11
3.1.3	Step 3: Compute the Eigenvectors and Eigenvalues	12
3.1.4	Step 4: Feature Vector Selection	12
3.1.5	Step 5: Projection (Recasting the Data)	12
3.2	Illustrative Example: Manual Verification	13
3.3	Verification on Real Dataset Subset	15
CHAPTER 4:	Applications of PCA	17
4.1	Image Processing	17
4.1.1	Pattern Recognition	17
4.1.2	Background Subtraction in Videos	17
4.1.3	Image Denoising	17
4.2	Finance	18
4.2.1	Yield Curve Analysis	18
4.2.2	Stress Testing Portfolios	18
4.2.3	Identifying Correlated Assets	18
4.3	Biology and Bioinformatics	18
4.3.1	Single-Cell RNA Sequencing (scRNA-seq)	18
4.3.2	Protein Structure Analysis	18
4.4	Machine Learning	19
4.4.1	Noise Reduction	19
4.4.2	Feature Engineering	19
4.4.3	Speeding Up Clustering	19
CHAPTER 5:	Case Study: Analysis of Vietnam Natural Disasters	20
5.1	Data Description	20
5.2	Methodology	21
5.2.1	Preprocessing Strategy: Log Transformation	21
5.3	Implementation in Python	21
5.4	Results and Interpretation	22
5.4.1	Scree Plot Analysis	22
5.4.2	Model Validation: Reconstruction Error	22
5.4.3	Biplot Analysis	23
CHAPTER 6:	CONCLUSION	24
CHAPTER 7:	REFERENCES	25

INTRODUCTION

Principal Component Analysis (PCA) is a dimensionality-reduction technique that transforms high-dimensional data into a smaller set of new variables called principal components, which capture the most important patterns in the data.

By focusing on directions of maximum variance, PCA helps simplify complex datasets, reduce noise, and improve visualization or model performance while preserving as much essential information as possible. It is widely used in fields such as machine learning, finance, image processing, and biology.

The objectives of this project include:

- Understanding the theoretical foundations of PCA.
- Applying matrix operations to solve real-world problems.
- Using MATLAB to simulate and calculate results efficiently.

Chapter 1

OVERVIEW

1.1 Problem Definition: Data Dimensionality

1.1.1 Definition of Data Dimensionality

Data dimensionality refers to the number of features (variables or attributes) present in a dataset. For a dataset with n samples and p features, the dimensionality is defined as p .

1.1.2 The Curse of Dimensionality

High-dimensional data presents several challenges, often referred to as the "Curse of Dimensionality". These challenges include:

- **Exponential increase in volume:** As dimensions increase, the data becomes sparse.
- **Distance metrics:** Distances between data points become less meaningful in high-dimensional space.
- **Computational cost:** High dimensionality leads to increased computational cost and memory usage.
- **Overfitting:** There is a higher risk of overfitting in machine learning models.
- **Visualization:** It is difficult for humans to visualize data beyond 3 dimensions.

1.1.3 Why Dimensionality Reduction is Needed

Dimensionality reduction is crucial to remove redundant or noisy features and retain the most important information. It helps to speed up algorithms, improve model generalization, and enable better visualization of complex datasets.

1.1.4 Types of Dimensionality Reduction Techniques

Techniques are generally categorized into:

- **Linear methods:** Such as Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA).
- **Non-linear methods:** Such as t-SNE, UMAP, and Autoencoders.
- **Justification for choosing PCA:** For this study, we selected PCA over non-linear methods like t-SNE because **interpretability** is our priority. While t-SNE is excellent for clustering, it distorts the feature space, making it impossible to tell which original variables (e.g., "Deaths" vs "Damage") caused the separation. PCA preserves the linear weights (loadings), allowing us to construct a Biplot that explicitly links disaster severity to specific economic or human costs.

1.2 Introduction to PCA

1.2.1 What is Principal Component Analysis?

Principal Component Analysis (PCA) is a linear transformation technique that projects high-dimensional data onto a new coordinate system known as principal components. In this new system:

- The **first principal component** captures the direction of maximum variance.
- Each **subsequent component** captures the maximum remaining variance while being orthogonal to the previous ones.

1.2.2 Historical Background

PCA was invented by Karl Pearson in 1901 and later developed by Harold Hotelling in 1933 as a statistical tool.

1.2.3 Key Intuition

The key intuition behind PCA is to find the "best" lower-dimensional subspace that preserves as much variability (information) as possible from the original dataset.

1.2.4 Illustrative Example

Consider a 2D dataset, such as the height versus weight of people. PCA rotates the axes to align with the direction of the greatest spread (variance) of the data.

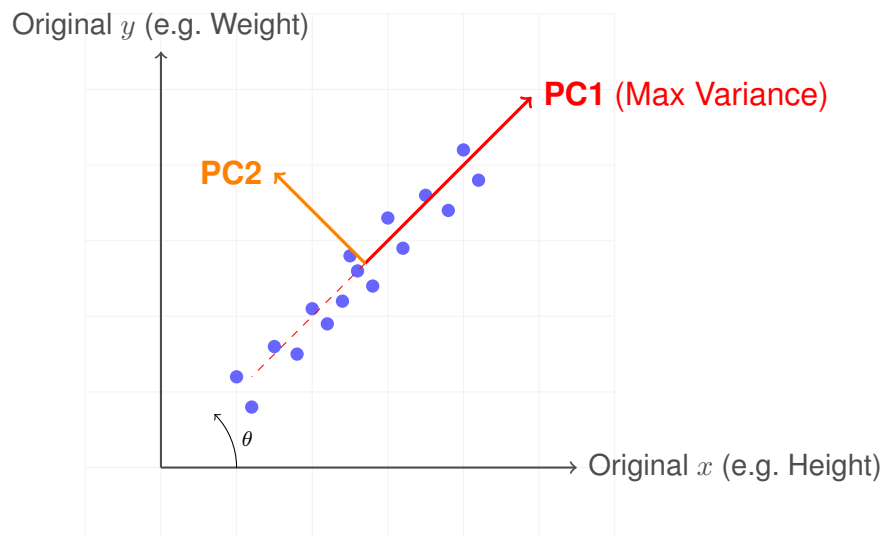


Figure 1.1: Visualizing PCA: The red axes (PC_1, PC_2) are rotated to align with the data's greatest spread, distinct from the original x, y axes.

1.2.5 Relationship with Other Methods

PCA is closely related to Singular Value Decomposition (SVD) and can be viewed as a special case of factor analysis.

1.3 Objectives, Advantages, and Disadvantages

1.3.1 Main Objectives

The primary goals of applying PCA are:

- Maximize preserved variance while minimizing the number of dimensions.
- Decorrelate features (principal components are orthogonal and uncorrelated).
- Enable visualization of high-dimensional data (e.g., reducing to 2D or 3D).
- Improve performance and reduce overfitting in downstream machine learning tasks.
- Reduce noise by discarding low-variance components.

1.3.2 Advantages

PCA offers several significant benefits:

- **Efficiency:** It is computationally efficient and scalable, especially with SVD-based implementations.

- **Unsupervised:** It does not require labeled data.
- **Deterministic:** The same input always yields the same result.
- **Accessibility:** It is widely available in libraries such as scikit-learn, MATLAB, and R.
- **Foundation:** It has a strong mathematical foundation and interpretability regarding variance.

1.3.3 Disadvantages

Despite its utility, PCA has limitations:

- **Linearity Assumption:** PCA assumes linear relationships and fails to capture non-linear structures (manifold learning or Kernel PCA may be required).
- **Interpretability:** Principal components are linear combinations of original features, which can be semantically hard to interpret.
- **Scaling Sensitivity:** Features must be standardized beforehand as PCA is sensitive to the scale of data.
- **Outliers:** PCA is sensitive to outliers, which can dominate the variance.
- **Information Definition:** It assumes variance is the best measure of information, which is not always true for classification tasks.

Chapter 2

Mathematical Foundations

This chapter establishes the statistical and algebraic framework necessary for understanding Principal Component Analysis (PCA), including expectation, covariance, and eigen-decomposition.

2.1 Statistical Features

2.1.1 Expectation

The expectation of a random variable X , denoted $E[X]$, is the theoretical long-run average value of X . It represents the center or "balance point" of the distribution.

For a Discrete Random Variable: If X takes values $x_1, x_2, \dots, x_n$ with probabilities $p_1, p_2, \dots, p_n$:

$$E[X] = \sum_{i=1}^n x_i p_i \quad (2.1)$$

For a Continuous Random Variable: If X has a probability density function $f(x)$:

$$E[X] = \int_{-\infty}^{\infty} x f(x) dx \quad (2.2)$$

2.1.2 Variance

Variance measures how spread out the values of a random variable are around the mean. A higher variance means the data is more dispersed.

Formula:

$$\text{Var}(X) = E[(X - E[X])^2] \quad (2.3)$$

This is the expected (average) squared distance from the mean.

Equivalent Computational Formula:

$$\text{Var}(X) = E[X^2] - (E[X])^2 \quad (2.4)$$

2.1.3 Standard Deviation

Standard deviation is the square root of variance:

$$\sigma = \sqrt{\text{Var}(X)} \quad (2.5)$$

2.2 Covariance

2.2.1 Definition (Scalar)

For integrable random variables X and Y :

$$\text{Cov}(X, Y) := E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]E[Y] \quad (2.6)$$

Note that the variance of X is simply the covariance of X with itself:

$$\text{Var}(X) = \text{Cov}(X, X)$$

2.2.2 Properties

- **Symmetry:** $\text{Cov}(X, Y) = \text{Cov}(Y, X)$.

- **Bilinearity:** Covariance is bilinear in both arguments:

$$\text{Cov}(aX + bY, Z) = a\text{Cov}(X, Z) + b\text{Cov}(Y, Z)$$

- **Relation to Independence:** If X and Y are independent (and integrable), then $\text{Cov}(X, Y) = 0$.

– *Note:* The converse is false; covariance zero does **not** imply independence in general.

- **Cauchy–Schwarz Bound:**

$$|\text{Cov}(X, Y)| \leq \sqrt{\text{Var}(X)\text{Var}(Y)}$$

This yields the **correlation coefficient** $\rho_{X,Y}$:

$$\rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sqrt{\text{Var}(X)\text{Var}(Y)}}, \quad |\rho| \leq 1$$

2.2.3 Covariance Matrix (Multivariate)

Let $X = (X_1, \dots, X_n)^\top$ be a random vector with finite second moments. Define the **covariance matrix** Σ :

$$\Sigma := \text{Cov}(X) = E[(X - \mu)(X - \mu)^\top], \quad \text{where } \mu = E[X] \quad (2.7)$$

The entries are given by $\Sigma_{ij} = \text{Cov}(X_i, X_j)$.

Key Facts:

- Σ is symmetric: $\Sigma^\top = \Sigma$.
- Σ is positive semidefinite (PSD): For any vector $a \in \mathbb{R}^n$:

$$a^\top \Sigma a = \text{Var}(a^\top X) \geq 0$$

- If all components have finite variance, then Σ is finite.

2.2.4 Conditional Covariance and Law of Total Covariance

- **Law of Total Expectation:** $E[X] = E[E[X|Y]]$.
- **Law of Total Covariance:**

$$\text{Cov}(X, Z) = E[\text{Cov}(X, Z|Y)] + \text{Cov}(E[X|Y], E[Z|Y]) \quad (2.8)$$

- **Law of Total Variance** (Special case):

$$\text{Var}(X) = E[\text{Var}(X|Y)] + \text{Var}(E[X|Y])$$

2.2.5 Whitening / Decorrelation

Given a positive definite covariance matrix Σ , one can compute a **whitening matrix** W such that:

$$W\Sigma W^\top = I$$

Common choices for W :

- $W = \Sigma^{-1/2}$ (via eigen-decomposition).
- $W = L^{-1}$ (via Cholesky decomposition $\Sigma = LL^\top$).

Whitening removes covariances (resulting in an identity covariance matrix) and is often used as preprocessing for algorithms like ICA.

2.2.6 Difference Between Diagonal and Off-Diagonal Values

A covariance matrix summarizes how pairs of features vary together. For features $X_1, \dots, X_n$:

$$\Sigma = \begin{pmatrix} \text{Cov}(X_1, X_1) & \cdots & \text{Cov}(X_1, X_n) \\ \vdots & \ddots & \vdots \\ \text{Cov}(X_n, X_1) & \cdots & \text{Cov}(X_n, X_n) \end{pmatrix}$$

1. Diagonal Values (Variances): Entries $\text{Cov}(X_i, X_i) = \text{Var}(X_i)$ describe the spread of a single dimension.

- **Large variance:** The feature values vary significantly.

- **Small variance:** The feature values are tightly clustered around the mean.

2. Off-Diagonal Values (Covariances): Entries $\text{Cov}(X_i, X_j)$ where $i \neq j$ describe how two features vary together.

- **Positive:** When X_i increases, X_j tends to increase.
- **Negative:** When X_i increases, X_j tends to decrease.
- **Zero:** Features are uncorrelated.

These values reflect the shape and orientation of the data cloud in multivariate space.

2.3 Eigenvalues and Eigenvectors

2.3.1 Algebraic Definition

For a square $n \times n$ matrix A , a scalar λ is an **eigenvalue** and a non-zero vector v is an **eigenvector** if:

$$Av = \lambda v \quad (2.9)$$

Equivalently, λ is a root of the characteristic polynomial:

$$p_A(\lambda) := \det(A - \lambda I) = 0$$

Multiplicity:

- **Algebraic multiplicity:** Multiplicity of λ as a root of $p_A(\lambda)$.
- **Geometric multiplicity:** Dimension of the nullspace $\ker(A - \lambda I)$ (number of linearly independent eigenvectors).
- Always $1 \leq \text{geom mult} \leq \text{alg mult}$.

2.3.2 Spectral Properties

- **Trace:** $\text{tr}(A) = \sum \lambda_i$ (sum of eigenvalues).
- **Determinant:** $\det(A) = \prod \lambda_i$ (product of eigenvalues).
- **Real Symmetric Matrices:** All eigenvalues are real, and eigenvectors can be chosen to be orthonormal (Spectral Theorem).

2.3.3 Singular Value Decomposition (SVD)

For any $m \times n$ real matrix M , SVD writes:

$$M = U\Sigma V^T \quad (2.10)$$

Where U and V are orthogonal matrices, and Σ is a diagonal matrix with non-negative **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.

- The singular values of M are the square roots of the eigenvalues of $M^T M$ (or MM^T).
- SVD generalizes eigen-decomposition to non-square matrices and is numerically stable.

2.3.4 Eigen-decomposition of Covariance Matrices & PCA

The covariance matrix Σ is symmetric and positive semidefinite. Its eigen-decomposition is:

$$\Sigma = Q\Lambda Q^T \quad (2.11)$$

where Q is orthonormal and $\Lambda = \text{diag}(\lambda_1 \geq \lambda_2 \geq \dots \geq 0)$.

Relation to PCA:

- **Principal Components:** The eigenvectors corresponding to the largest eigenvalues.
- **Variance Explained:** The variance explained by the k -th principal component is equal to λ_k .
- PCA projects mean-centered data onto the first r eigenvectors to create a lower-dimensional representation that preserves maximal variance.

Chapter 3

The Principal Component Analysis Algorithm

3.1 Theoretical Framework

Principal Component Analysis (PCA) is a dimensionality reduction technique that transforms a high-dimensional dataset into a smaller set of uncorrelated variables called Principal Components, while retaining as much of the original variation as possible [1]. The algorithm follows a rigorous five-step process.

3.1.1 Step 1: Standardization (Data Scaling)

The first step is to standardize the range of the continuous initial variables so that each of them contributes equally to the analysis. If there are large differences between the ranges of initial variables, those variables with larger ranges will dominate over those with small ranges.

For example, a "Damage Cost" variable ranging in the millions will mathematically overpower a "Death Count" variable ranging in the tens [2]. Transforming the data to comparable scales prevents this problem. We typically do this using the Z-score normalization:

$$z = \frac{x - \mu}{\sigma} \quad (3.1)$$

Where x is the original value, μ is the mean, and σ is the standard deviation. This results in a dataset where every variable has a mean of 0 and a variance of 1 [2].

3.1.2 Step 2: Covariance Matrix Computation

The aim of this step is to understand how the variables of the input data set are varying from the mean with respect to each other. The covariance matrix (Σ) is a symmetric matrix that has as entries the covariances associated with all possible pairs of the initial variables [3].

$$\Sigma = \frac{1}{n-1} X^T X \quad (3.2)$$

Since the covariance of a variable with itself is its variance, the main diagonal contains the variances of each initial variable. High positive values indicate variables that increase together, while zero values indicate no relationship.

3.1.3 Step 3: Compute the Eigenvectors and Eigenvalues

Eigenvectors and eigenvalues are the linear algebra concepts that we need to compute from the covariance matrix in order to determine the principal components of the data [3].

- **Eigenvectors** (v): Determine the directions of the new feature space (where the data is most spread out).
- **Eigenvalues** (λ): Determine the magnitude (or variance) of the data along those new directions.

Mathematically, we solve the characteristic equation:

$$\det(\Sigma - \lambda I) = 0 \quad (3.3)$$

3.1.4 Step 4: Feature Vector Selection

Once eigenvectors are found, we order them by their eigenvalues, from highest to lowest ($\lambda_1 \geq \lambda_2 \dots$). This gives us the components in order of significance. The eigenvector with the highest eigenvalue is the First Principal Component (PC1) [1].

We then choose to keep the top k eigenvectors to form our feature vector W . This is the step where dimensionality reduction actually occurs, as we discard the components with low information (noise).

3.1.5 Step 5: Projection (Recasting the Data)

In the final step, we use the feature vector W formed in the previous step to reorient the data from the original axes to the ones represented by the principal components. This is done by matrix multiplication [3]:

$$Z = X \cdot W \quad (3.4)$$

The resulting matrix Z represents the original data compressed into fewer dimensions.

3.2 Illustrative Example: Manual Verification

To demonstrate the mathematical mechanics of the algorithm described above, we perform a manual calculation on a simplified dataset.

Problem: We have data for 3 students involving their study hours (Variable x) and their test scores (Variable y). We wish to reduce this 2D data into 1D using PCA.

The Dataset:

Table 3.1: Dataset for Manual Calculation Example

Student	Study Hours (x)	Test Score (y)
A	2	8
B	4	12
C	6	16

Execution

1. Standardization (Centering):

First, we calculate the means: $\bar{x} = 4$, $\bar{y} = 12$. We center the data by subtracting the means from each point:

$$X_{cen} = \begin{pmatrix} 2-4 & 8-12 \\ 4-4 & 12-12 \\ 6-4 & 16-12 \end{pmatrix} = \begin{pmatrix} -2 & -4 \\ 0 & 0 \\ 2 & 4 \end{pmatrix}$$

2. Covariance Matrix:

We calculate the covariance matrix Σ using the formula $\Sigma = \frac{1}{n-1} X^T X$. Since $n = 3$, we divide by 2.

$$\Sigma = \frac{1}{2} \begin{pmatrix} -2 & 0 & 2 \\ -4 & 0 & 4 \end{pmatrix} \begin{pmatrix} -2 & -4 \\ 0 & 0 \\ 2 & 4 \end{pmatrix} = \begin{pmatrix} 4 & 8 \\ 8 & 16 \end{pmatrix}$$

3. Eigenvalues:

We solve the characteristic equation $\det(\Sigma - \lambda I) = 0$:

$$\det \begin{pmatrix} 4-\lambda & 8 \\ 8 & 16-\lambda \end{pmatrix} = 0$$

$$(4-\lambda)(16-\lambda) - 64 = 0 \Rightarrow \lambda^2 - 20\lambda = 0$$

Solving for λ gives two results:

- $\lambda_1 = 20$ (This is PC1, containing all the information).
- $\lambda_2 = 0$ (This is PC2, containing no information).

4. Eigenvectors:

We find the eigenvector for the largest eigenvalue ($\lambda_1 = 20$) by solving the system $(\Sigma - 20I)v = 0$:

$$\begin{pmatrix} -16 & 8 \\ 8 & -4 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

This simplifies to: $-16x + 8y = 0 \Rightarrow 2x = y$. We choose a simple vector that fits this rule: $v = [1, 2]^T$.

To make it a unit vector (length of 1), we divide by $\sqrt{1^2 + 2^2} = \sqrt{5}$:

$$u_1 = \begin{pmatrix} 1/\sqrt{5} \\ 2/\sqrt{5} \end{pmatrix} \approx \begin{pmatrix} 0.45 \\ 0.89 \end{pmatrix}$$

5. Projection:

We project the centered data onto our new axis (u_1) to get the final reduced data (Z):

$$Z = X_{cen} \cdot u_1 = \begin{pmatrix} -2 & -4 \\ 0 & 0 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 0.45 \\ 0.89 \end{pmatrix} = \begin{pmatrix} -4.47 \\ 0 \\ 4.47 \end{pmatrix}$$

Conclusion: We successfully reduced the data from 2 coordinates (Hours, Score) to just 1 coordinate (Z) while capturing 100% of the variance.

3.3 Verification on Real Dataset Subset

To further validate our approach, we perform the same manual calculation on a subset of the actual *Vietnam Natural Disaster* dataset. We selected three historical events from the 1960s where both human and economic impacts were recorded to observe how the large variance in "Damage" influences the principal components.

The Subset (Raw Data):

Table 3.2: Subset of Vietnam Disasters for Manual Verification

Event	Type	Deaths (x)	Damage ('000 \$) (y)
1	Typhoon Iris (1964)	7,000	471,770
2	Riverine Flood (1966)	31	90,163
3	Fire Accident (1967)	3	4,387

Step-by-Step Execution

1. Calculation of Means:

First, we compute the average for each feature:

$$\bar{x} = \frac{7000 + 31 + 3}{3} \approx 2,344.67$$

$$\bar{y} = \frac{471770 + 90163 + 4387}{3} \approx 188,773.33$$

2. Centering the Data ($X - \mu$):

We subtract the mean from each data point to center the dataset at (0,0):

$$X_{cen} = \begin{pmatrix} 4655.33 & 282996.67 \\ -2313.67 & -98610.33 \\ -2341.67 & -184386.33 \end{pmatrix}$$

3. Covariance Matrix (Σ):

We calculate the covariance matrix using $\Sigma = \frac{1}{n-1} X_{cen}^T X_{cen}$ (with $n = 3$):

$$\Sigma \approx \begin{pmatrix} 1.63 \times 10^7 & 9.89 \times 10^8 \\ 9.89 \times 10^8 & 6.19 \times 10^{10} \end{pmatrix}$$

Observation: The variance of Variable y (Damage) is approximately 6.19×10^{10} , which is significantly larger than the variance of x (Deaths). This mathematically confirms why standardization (Log Transformation/Scaling) is critical for this dataset; otherwise, "Damage" would completely dominate the analysis.

4. Eigenvalues and Eigenvectors:

Solving the characteristic equation $\det(\Sigma - \lambda I) = 0$ yields two eigenvalues:

- $\lambda_1 \approx 6.19 \times 10^{10}$ (Accounts for $> 99\%$ of the variance).

- $\lambda_2 \approx 4.64 \times 10^5$ (Accounts for $< 1\%$ of the variance).

The eigenvector corresponding to PC1 is approximately $v_1 = [-0.016, -0.999]^T$. The value -0.999 indicates that PC1 is almost entirely aligned with the "Damage" axis.

5. Projection:

Finally, we project the centered data onto the new PC1 axis ($Z = X_{cen} \cdot v_1$):

$$Z \approx \begin{pmatrix} -283,035 \\ 98,635 \\ 184,400 \end{pmatrix}$$

This step successfully reduces our 2-dimensional data (Deaths, Damage) into a 1-dimensional representation (Z) while preserving the vast majority of the variance found in the original subset.

Chapter 4

Applications of PCA

Principal Component Analysis is a versatile tool used across various domains to manage high-dimensional data. This chapter explores its critical applications in Image Processing, Finance, Biology, and Machine Learning.

4.1 Image Processing

4.1.1 Pattern Recognition

Images are inherently high-dimensional; for instance, a small 28×28 pixel image contains 784 features. PCA reduces these features to a small number (typically 40–60) while preserving the main structure. The principal components often represent fundamental shapes such as vertical strokes, curves, or loops. Machine learning models trained on these reduced images run faster, generalize better, and require less storage. This makes PCA a staple in character recognition tasks.

4.1.2 Background Subtraction in Videos

Videos often consist of static backgrounds (walls, roads) and moving foreground objects. PCA can identify the background because it appears in almost every frame, resulting in very high variance stability. Conversely, moving objects change rapidly and are captured in low-variance components. By subtracting the PCA-reconstructed background, systems can isolate moving objects for tracking, motion detection, and surveillance.

4.1.3 Image Denoising

Random noise in images is typically uncorrelated and appears in the low-variance components (the "tail" of the eigenvalue spectrum). By reconstructing an image using only the top principal components, we preserve edges and large structures while filtering out the fine-grain noise. This technique is effective in medical imaging and astronomy.

4.2 Finance

4.2.1 Yield Curve Analysis

In fixed income markets, the yield curve describes interest rates for bonds of different maturities. PCA reveals that complex curve movements are driven by three main components:

- **Level (PC1):** An overall up/down shift of interest rates.
- **Slope (PC2):** The difference between short-term and long-term rates.
- **Curvature (PC3):** How "bowed" the curve is.

Traders and central banks use these factors to model risk and understand economic expectations.

4.2.2 Stress Testing Portfolios

Regulatory frameworks require banks to test portfolios against extreme events. PCA identifies components representing major risk sources. By "shocking" these components (e.g., forcing PC1 to jump by 5 standard deviations), analysts can simulate realistic stress scenarios that capture how markets move together, providing more accurate risk estimates than shocking individual assets.

4.2.3 Identifying Correlated Assets

Markets contain assets that behave similarly due to hidden factors. PCA applied to return matrices reveals which assets cluster together. This helps investors build diversified portfolios and avoid accidental overexposure to specific hidden risks (e.g., a common factor driving all commodities).

4.3 Biology and Bioinformatics

4.3.1 Single-Cell RNA Sequencing (scRNA-seq)

Modern biology datasets are massive, often containing 20,000+ gene expression values per cell. PCA compresses this space into 30–50 components representing major cell states and biological processes. This allows researchers to cluster cells into distinct types (e.g., T-cells, Neurons) which would be impossible in the noisy, high-dimensional raw space.

4.3.2 Protein Structure Analysis

Proteins are dynamic molecules. Simulations of their movement generate high-dimensional atomic coordinate data. PCA identifies dominant motions, such as the opening/closing of

protein domains or twisting motions, helping researchers understand how proteins fold and interact with drugs.

4.4 Machine Learning

4.4.1 Noise Reduction

Real-world data from sensors or IoT devices often contains noise. Since noise usually contributes very little variance, it is relegated to the lowest principal components. By dropping these components, PCA acts as an automated filter, cleaning the input data and improving the robustness of downstream models.

4.4.2 Feature Engineering

Raw features often contain redundant information. PCA creates new composite features that capture the strongest signals and remove collinearity. For tasks like credit scoring or churn prediction, these uncorrelated features help algorithms learn more efficiently.

4.4.3 Speeding Up Clustering

Clustering algorithms like K-Means or DBSCAN struggle in high dimensions due to computational cost and the breakdown of distance metrics. Reducing thousands of features to a few dozen principal components makes clustering faster, more accurate, and more stable.

Chapter 5

Case Study: Analysis of Vietnam Natural Disasters

In this chapter, we apply Principal Component Analysis (PCA) to a real-world dataset to demonstrate its effectiveness in reducing dimensionality and identifying patterns in complex data.

5.1 Data Description

This analysis utilizes the *Vietnam Natural Disaster* dataset, which records historical disaster events from 1953 to 2023. To analyze the severity and dimensionality of these events, we selected four key numerical variables that measure impact:

- **Total Deaths:** The number of fatalities per event.
- **No. Injured:** The number of people injured.
- **Total Affected:** The total population affected (homeless, evacuated, etc.).
- **Total Damage:** The economic cost adjusted for inflation (in thousands of US Dollars).

Table 5.1: Sample of the Vietnam Natural Disaster Dataset (Raw Data)

Event ID	Total Deaths	Injured	Affected	Damage Adj ('000 US\$)
1953-0019-VNM	1,000	0	0	0
1956-0050-VNM	56	0	0	0
1964-0004-VNM	598	0	10,848	0
1964-0063-VNM	7,000	0	700,000	471,770
1964-0102-VNM	400	0	0	0

Note: Top 5 rows from the dataset. Missing values (NaN) were filled with 0 during preprocessing.
Event 1964-0063-VNM (Typhoon Iris) shows the high variance in the data.

5.2 Methodology

5.2.1 Preprocessing Strategy: Log Transformation

Real-world disaster data is typically highly skewed; a single catastrophic typhoon may cause billions of dollars in damage, while hundreds of smaller floods cause significantly less. If raw data were used directly, the few "outlier" events would dominate the variance, causing PCA to ignore the patterns in common disasters.

To resolve this, we applied a **Logarithmic Transformation** ($x' = \log(x + 1)$) to all variables. This compresses the scale of extreme values, bringing the "billion-dollar" events closer to the average events statistically. This allows PCA to analyze the *structure* of the data rather than just the magnitude.

5.3 Implementation in Python

We performed the analysis using the Python `scikit-learn` library. The core steps included loading the data, applying the log transformation, standardizing the values (Z-score), and computing the principal components.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

# 1. LOAD DATA
df = pd.read_csv('disaster_in-vietnam.csv')
features = ['Total Deaths', 'No. Injured', 'Total Affected',
            'Total Damage, Adjusted ('000 US$)']
X = df[features].fillna(0)

# 2. LOG TRANSFORMATION
# We use log(1+x) to handle skewness
X_log = np.log1p(X)

# 3. STANDARDIZATION
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_log)

# 4. APPLY PCA
pca = PCA(n_components=2)
principalComponents = pca.fit_transform(X_scaled)

# 4.5 VALIDATION (Reconstruction Error)
X_reconstructed = pca.inverse_transform(principalComponents)
loss = np.linalg.norm(X_scaled - X_reconstructed) / np.linalg.norm(X_scaled)
print(f"Reconstruction Loss: {loss:.4f}")

# 5. RESULTS
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
```

Listing 5.1: Python Code for PCA Analysis

5.4 Results and Interpretation

5.4.1 Scree Plot Analysis

The Scree Plot below (Figure 5.1) illustrates the variance explained by each component.

- **PC1 (First Component):** Captures approximately **47.6%** of the total information.
- **PC2 (Second Component):** Captures approximately **22.8%** of the total information.

Together, these two dimensions preserve over **70%** of the dataset's original variability, successfully reducing the data from 4 dimensions to 2.

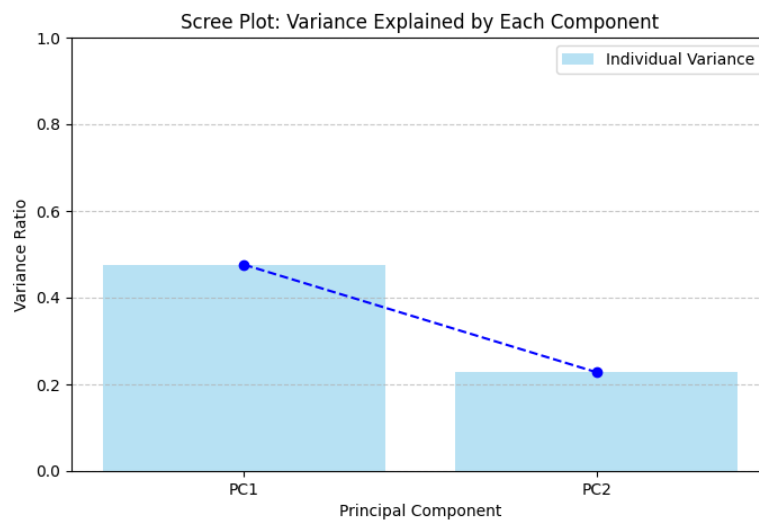


Figure 5.1: Scree Plot showing the variance explained by the first two components.

5.4.2 Model Validation: Reconstruction Error

To validate the quality of our dimensionality reduction, we calculated the **Reconstruction Error**. This metric measures the deviation between the original data (X) and the data reconstructed from the two principal components (X_{approx}). It is defined as the normalized Frobenius norm of the difference:

$$\text{Error} = \frac{\|X - X_{approx}\|_F}{\|X\|_F} \quad (5.1)$$

Our analysis yielded a reconstruction loss of **0.5439**. This result is mathematically consistent with our Scree Plot analysis. Since our two principal components capture approximately **70.4%** of the total variance, the remaining **29.6%** accounts for the information loss ($\sqrt{0.296} \approx 0.54$). This confirms that while some fine-grained detail is lost, the 2D projection successfully captures the dominant structural relationships of the disaster dataset.

5.4.3 Biplot Analysis

The PCA Biplot (Figure 5.2) visualizes both the disaster events (points) and the original variables (vectors).

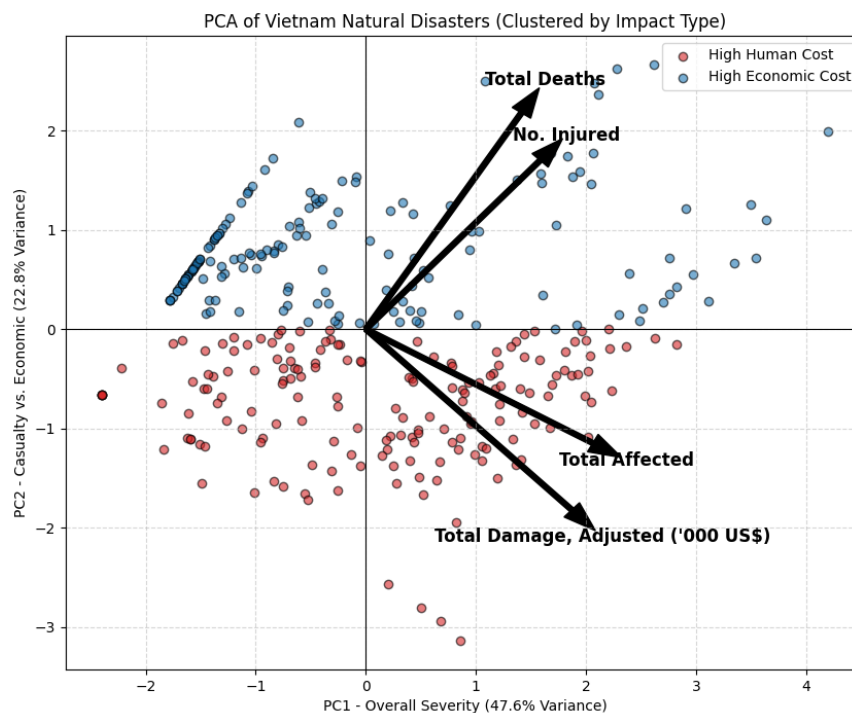


Figure 5.2: PCA Biplot of Vietnam Natural Disasters (Log-Transformed).

Interpretation of Axes:

- **PC1 (Horizontal Axis) - Overall Severity:** All four feature vectors point to the right. This indicates that PC1 represents the general magnitude of a disaster. Events further to the right are historically the most catastrophic in terms of both human and economic cost.
- **PC2 (Vertical Axis) - Human vs. Economic Cost:** This axis reveals a trade-off. "Total Deaths" and "Injured" vectors point downwards, while "Total Damage" points upwards.
 - *Top Quadrant:* Events with high economic damage but fewer casualties (modern storms).
 - *Bottom Quadrant:* Events with high casualties (deadlier historical events).

Chapter 6

CONCLUSION

In conclusion, PCA is a powerful and practical method for reducing dimensionality, simplifying complex datasets, and revealing their underlying structure. By capturing the directions of greatest variance, it allows us to preserve essential information while removing noise and redundancy.

Whether applied to machine learning, financial analysis, image processing, or biological data, PCA provides a foundation for clearer insights and more efficient models. Understanding how PCA works—and when to use it—helps us analyze data more effectively and make better, data-driven decisions.

Chapter 7

REFERENCES

1. Abdi, H., & Williams, L. J. (2010). Principal component analysis. *WIREs Computational Statistics*, 2(4), 433–459.
2. Bishop, C. M. (2006). *Pattern recognition and machine learning* (Chapter 12). Springer.
3. Duda, R. O., Hart, P. E., & Stork, D. G. (2012). *Pattern classification* (2nd ed., Section 3.8). Wiley.
4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed., Chapter 14.5). Springer.
5. Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.
6. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An introduction to statistical learning: With applications in R* (2nd ed., Chapter 12). Springer.
7. Jolliffe, I. T. (2002). *Principal component analysis* (2nd ed.). Springer.
8. Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, Series 6, 2(11), 559–572.
9. Shlens, J. (2014). A tutorial on principal component analysis. *arXiv preprint arXiv:1404.1100*.
10. Strang, G. (2023). *Linear algebra and learning from data* (Chapter 7). Wellesley-Cambridge Press.
11. VanderPlas, J. (2016). *Python data science handbook: Essential tools for working with data*. O'Reilly Media.
12. Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
13. Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.
14. Shlens, J. (2005). "Notes on principal component analysis," Google Research. [Online]. Available: https://www.cs.princeton.edu/picasso/mats/PCA-Tutorial-Intuition_josh.pdf

15. "Principal Component Analysis (PCA) explained," Towards Data Science. [Online]. Available: <https://towardsdatascience.com/pca-explained>